Project Graph

设计与生成规范

v1.5 · 关系感知布局版

核心原则

先分区，后摆点；先留白，后连线；先语义，后装饰；先可读，后紧凑。

不是“块之间不重叠”就够了，而是块、线、标签、Section 与 Hyperedge 边界都必须拥有安全活动空间。

配套：Project Graph Architect Skill v1.0.0

目标格式：PRG metadata 2.7.0

0. 文档定位

这份规范不是为了规定一种固定画风，而是把 Project Graph 的语义建模、原生原语选择、布局、连线路由、颜色、校验和交付过程统一成一套可复用的工程方法。Skill 是执行真源，本规范是人类可读的设计依据与验收标准。

> **交付目标**
> 给定一段项目、思想、工作流、系统或比较关系，最终产物应是：语义正确、原语选择合理、可读、可继续编辑、Project Graph 可直接打开的 `.prg`。

- 1. 设计目标与方法论
- 2. Project Graph 原生原语选择
- 3. 关系感知布局模型
- 4. 连线路由与标签
- 5. Section / Hyperedge / 子图
- 6. 颜色、对比度与视觉层级
- 7. PRG 2.7 工程格式
- 8. 生成工作流与二次松弛
- 9. QA 与失败判定
- 10. Skill 行为规范
- 11. 版本演进与后续计划

1. 设计目标与方法论

1.1 四个优先级

语义优先：先明确“这是什么、与谁有什么关系”，再考虑坐标。图在没有美化前就应当语义自洽。

可读优先：位置不需要节省。图更大可以接受，关系线、标签、节点叠在一起不能接受。

原生优先：能用 Project Graph 的原生 Section、ArcEdge、Hyperedge、SyncAssociation、ReferenceBlock 表达，就不要退化成纯装饰。

验证优先：能打开不等于合格。必须经过结构校验、几何审计和至少一轮 post-routing cleanup。

1.2 认知与人体工学目标

- 用户第一眼先看到大结构，而不是先看到乱线。
- 重要阅读路径尽量单向、稳定、少折返。
- 颜色承担语义，但不能牺牲深浅主题下的易读性。
- 如果用户需要停下来猜“这串字属于哪根线”，布局即失败。
- 留白既是美学空间，也是关系表达的交通空间。

2. Project Graph 原生原语选择

原语	适用	不适用	设计提醒
TextNode	实体、概念、状态、原则、规则	把每句说明都做节点	标题短；长解释进入 Details
LineEdge	普通单向语义关系	双向关系的两条边叠成一条	在线上写明确关系词；保留箭头前后可见线段
ArcEdge	反馈、回流、反向、绕障	主链全部弯曲	主链直、回流弯；长距离优先外圈
Hyperedge	真实多元共同关系 / 群组	装饰框	群组备注应优先通过成员位置和 padding 留空
Section	可折叠语义模块	纯背景色块	标题下方必须留标题带
SyncAssociation	同一概念的多个视图实例	普通相似 / 相关	只同步真正相同的概念
ReferenceBlock	独立复杂子图	为了省空间随便拆图	适合可独立维护、打开、复用的模块
Details	长解释、证据、边界条件	核心身份与关系	用于降低画布文字密度

2.1 一条关键原则

不要为了修布局破坏语义

如果一段文字本来是“整个 Hyperedge 的备注”，优先通过移动成员、扩大 padding、移动整组来制造空白区；不要第一反应把它变成一个假的独立节点。

3. 关系感知布局模型

3.1 节点不是只有一个矩形

Content Box — 节点真实边界，承载文字。

Visual Halo — 视觉呼吸区，无关内容不应紧贴。

Routing Exclusion Zone — 边与标签默认不可进入的保护区，终止于该节点的边除外。

Traffic Context — 由局部边密度、标签长度、回流、Section 边界动态扩张的空间。

3.2 动态间距而非统一 gap

推荐思路

required spacing = base spacing + label budget + edge-density budget + return-path budget + section padding

- 基础节点间距可从 140–220 world units 起步。
- 两行关系词需要额外纵向/横向预算。
- 高密度 fan-out 区域要继续增加 100–220 的活动空间。

- 双向或 feedback 至少给回流单独预留 180–320 的通道。
- 大 Section 之间要有独立模块安全带，不要只让外框勉强不相交。

3.3 模块车道

- 总标题 / 总原则带
- 主模块带
- 主流程车道
- 旁支状态车道
- 事件车道
- 投影 / 映射车道
- 回流 / 反馈车道
- 关系标签带
- 外圈长距离路由走廊
- ReferenceBlock / 子图区

4. 连线路由与标签

4.1 四种关系几何

主链：尽量直、单向、方向一致。

旁支：从主链侧向分出，不占主链关系词空间。

回流：使用 ArcEdge，长距离回流优先走上沿、下沿或外圈。

投影：当前状态到另一视图的映射单独占 lane。

4.2 标签不是“免费空间”

- 关系词要在布局前预算宽度和行数。
- 必要时主动换行，而不是压成一条长串。
- 箭头前后都要留出可辨识线段。
- 重复关系词可稀疏化：布局已经表达模式时，不必每条都写。
- 复杂说明放 Details，不要让边标签承担段落。

4.3 长距离回边

> **默认规则**
>
> 正向链走中间，回流链走外圈；有连续空走廊就用空走廊。

5. Section、 Hyperedge 与子图

5.1 Section

- Section 是语义模块，不是单纯装饰边框。

- 标题下面必须有一条空的标题带。
- Section padding 要随内部关系复杂度动态增长。
- 不同 Section 的“活动空间”也不能重叠。

5.2 Hyperedge

- Hyperedge 应表示真实共同关系或包裹关系。
- 如果备注说明整个框，优先保留为 Hyperedge 的备注。
- 备注冲突时，先移动成员 → 增大 padding → 移动整个群组 → 最后才考虑改变表示方式。
- Hyperedge 边界也会占空间，不能穿过其它板块的核心内容区。

5.3 ReferenceBlock

适合拆成子图的情形：

- 模块本身足够复杂，主图只需要摘要；
- 模块可以独立维护和复用；
- 不同设备、子系统、阶段有清晰边界；
- 主图继续扩张会破坏阅读层级。

6. 颜色、对比度与视觉层级

6.1 节点色与边色必须分工

> **重要**
>
> 节点填充色负责语义与层级；边 / 标签颜色负责关系可读性。不要把暗色节点填充直接拿去做暗色画布上的关系标签。

6.2 主题安全

- 边色采用中等明度、中等饱和度、高 alpha。
- 避免暗棕、暗紫、暗绿在黑底上消失。
- 也避免过浅颜色在白底上发虚。
- 同一模块保持同一色相家族，靠透明度、尺寸、字重表达层级。

语义家族	推荐用途	边色示例
蓝	入口 / 归属 / 规范源	(89,140,205,0.96)
青绿	手势 / 协作 / 可达性	(71,172,150,0.96)
紫	输入 / 关系 / 对象结构	(177,121,220,0.96)
赭金	生命周期 / 自动化 / 过程	(196,141,84,0.96)
绿	反馈 / 性能 / 校准	(92,169,130,0.96)
中性灰	跨模块通用关系	(165,165,165,0.96)

7. PRG 2.7 工程格式

当前 Skill 的默认输出目标是 metadata version `2.7.0`。`.prg` 是 ZIP 容器，核心条目如下：

- stage.msgpack
- tags.msgpack
- reference.msgpack
- metadata.msgpack
- 可选 README.md、attachments/...、thumbnail.png

> **引用方式**
> stage 内对象间引用使用 {"$":"/<stage-index>"}。生成时必须保证索引存在并在最终 stage 长度范围内。

推荐使用 Skill 包中的 `scripts/prg27.py`，不要在每次任务里重新凭记忆手写序列化结构。

8. 生成工作流与二次松弛

1. 理解用户目标，抽取实体、状态、关系、反馈、重复视图与子系统。
2. 选择原生原语。
3. 规划模块与车道，而不是直接写坐标。
4. 根据标签长度和关系密度分配动态间距。
5. 放置节点。
6. 布置主链、旁支、回流、投影。
7. 设置 Section / Hyperedge / Sync / ReferenceBlock。
8. 应用主题安全配色。
9. 生成 PRG。
10. 运行 validate_prg.py。
11. 运行 layout_audit.py。
12. 做至少一次 post-routing cleanup / relaxation。
13. 重新生成并再次校验。
14. 交付主 PRG 与必要的子图目录 / ZIP。

8.1 二次松弛可采取的动作

- 移动单个节点
- 移动整个 Section
- 增大横向 / 纵向 gap
- 改变边的出入口侧
- 调整 ArcEdge offset
- 调整 textPosition
- 把长回边搬到外圈
- 增大 Hyperedge padding

- 减少重复标签
- 把长解释转移到 Details

9. QA 与失败判定

9.1 结构失败

- 缺少核心 MsgPack 条目
- metadata 版本异常且未说明
- stage 引用越界或格式错误
- ReferenceBlock 子文件未随包交付

9.2 语义失败

- 用 Hyperedge 只是为了画框
- 把不同概念错误同步
- 关系词无法说明两个节点之间到底是什么关系
- 大量解释塞进画布导致信息层级失控

9.3 布局失败

- 线压块
- 关系词压块
- 关系词互相重叠
- 边与箭头被文字吃掉，只剩“飘着的一行字”
- 大板块活动空间重叠
- 有空走廊却仍横穿核心节点
- Hyperedge 备注压在成员节点上
- Section 标题与第一排内容争空间

9.4 视觉失败

- 边色在黑底或白底上不可辨
- 过多色相导致模块难以记忆
- 次级元素比主链更抢眼
- 高饱和霓虹色造成长期观看疲劳

10. Project Graph Architect Skill 行为规范

- 默认输出真实 `.prg`，除非用户明确指定其它格式。
- 不把 Mermaid / PNG 当作 `.prg` 的替代品。
- 把公开 Project Graph 仓库视为只读参考；未经用户明确要求，不做 Issue、PR、评论、提交、上传等写操作。

- 生成复杂图时优先使用原生原语，不退化成全部 TextNode + 直线。
- 用户指出布局问题后，应把有效经验上升为通用规则，而不是只修该坐标。
- Skill 是执行真源；规范文档作为说明与验收，不应发展出另一套冲突规则。

11. 版本演进与后续计划

版本	关键升级	沉淀
v1.0	线可标注、基础 PRG 生成	语义边
v1.1–v1.2	连线留白、回边走外圈	边的活动空间
v1.3–v1.4	群组备注与大板块留白	先动布局再动结构
v1.5	关系感知布局、动态间距、二次松弛	节点 + 线 + 标签 + 群组边界联合布局
Skill v1.0.0	把上述规则编译进执行流程	可复用生成器 + 校验器 + 审计器

下一步

如果后续实测仍出现稳定可复现的冲突，应继续升级 Skill 的布局审计与自动松弛逻辑，而不是不断累积人工坐标特例。